NORAIZ RANA

Task #23

Topic

APIs Data Handling

REST & GraphQL basics

Date:

14 August 2025

SUBMITTED TO:

APPVERSE TECHNOLOGIES

Learning Report: Handling API Errors & Loading States in React

1. Introduction

When working with APIs in React, user experience depends on how we manage loading states and handle errors. Without them, users might think the app is frozen or broken. Proper feedback ensures a smooth and professional interface.

---

2. Loading State

A loading state shows that data is being fetched.
Example:

```
const [loading, setLoading] = useState(true);

useEffect(() => {
  fetch('https://api.example.com/data')
    .then(res => res.json())
    .then(() => setLoading(false));
}, []);
```

UI handling:

```
{loading ? <p>Loading...</p> : <Content />}
```

This prevents empty screens and improves clarity.

---

3. Error Handling

Errors can occur due to network issues, server errors, or wrong endpoints.
Example:

```
const [error, setError] = useState(null);

useEffect(() => {
  fetch('https://api.example.com/data')
    .then(res => {
      if (!res.ok) throw new Error('Failed to
fetch');
      return res.json();
    })
    .catch(err => setError(err.message))
    .finally(() => setLoading(false));
}, []);
```

We show clear, friendly error messages so users understand what happened.

---

4. Combined Usage

```
if (loading) return <p>Loading...</p>;
if (error) return <p
style={{color:'red'}}>Error: {error}</p>;
return <DataComponent />;
```

This ensures we cover all scenarios: waiting, failure, and success.

---

5. Best Practices

- Set `loading` before each fetch.
- Use `try...catch` with `async/await` for cleaner code.
- Always stop loading in `.finally()`.
- Provide human-friendly error text.
